# breast-cancer-data-analysis-1

November 26, 2025

## 1 STEP-1: EXPLORATORY DATA ANALYSIS

[90]:
```
# 1. IMPORTING ALL REQUIRED LIBRARIES
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix,␣
 ↪classification_report

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
```

[91]:
```
# STEP 2: SET DISPLAY OPTIONS
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', None)
```

[92]:
```
# STEP 3: LOAD THE DATASET
df = pd.read_csv("breast_cancer_data.csv")
```

[93]:
```
# Display the dataframe
df.head()
```

[93]:

| | id | diagnosis | radius_mean | texture_mean | perimeter_mean | area_mean | \ |
|---|---|---|---|---|---|---|---|
| 0 | 842302 | M | 17.99 | 10.38 | 122.80 | 1001.0 | |
| 1 | 842517 | M | 20.57 | 17.77 | 132.90 | 1326.0 | |
| 2 | 84300903 | M | 19.69 | 21.25 | 130.00 | 1203.0 | |
| 3 | 84348301 | M | 11.42 | 20.38 | 77.58 | 386.1 | |
| 4 | 84358402 | M | 20.29 | 14.34 | 135.10 | 1297.0 | |

| | smoothness_mean | compactness_mean | concavity_mean | concave points_mean | \ |
|---|---|---|---|---|---|
| 0 | 0.11840 | 0.27760 | 0.3001 | 0.14710 | |

```
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   symmetry_mean  fractal_dimension_mean  radius_se  texture_se  perimeter_se  \
0         0.2419                 0.07871     1.0950      0.9053         8.589
1         0.1812                 0.05667     0.5435      0.7339         3.398
2         0.2069                 0.05999     0.7456      0.7869         4.585
3         0.2597                 0.09744     0.4956      1.1560         3.445
4         0.1809                 0.05883     0.7572      0.7813         5.438

   area_se  smoothness_se  compactness_se  concavity_se  concave points_se  \
0   153.40       0.006399         0.04904       0.05373            0.01587
1    74.08       0.005225         0.01308       0.01860            0.01340
2    94.03       0.006150         0.04006       0.03832            0.02058
3    27.23       0.009110         0.07458       0.05661            0.01867
4    94.44       0.011490         0.02461       0.05688            0.01885

   symmetry_se  fractal_dimension_se  radius_worst  texture_worst  \
0      0.03003              0.006193         25.38          17.33
1      0.01389              0.003532         24.99          23.41
2      0.02250              0.004571         23.57          25.53
3      0.05963              0.009208         14.91          26.50
4      0.01756              0.005115         22.54          16.67

   perimeter_worst  area_worst  smoothness_worst  compactness_worst  \
0           184.60      2019.0            0.1622             0.6656
1           158.80      1956.0            0.1238             0.1866
2           152.50      1709.0            0.1444             0.4245
3            98.87       567.7            0.2098             0.8663
4           152.20      1575.0            0.1374             0.2050

   concavity_worst  concave points_worst  symmetry_worst  \
0           0.7119                0.2654          0.4601
1           0.2416                0.1860          0.2750
2           0.4504                0.2430          0.3613
3           0.6869                0.2575          0.6638
4           0.4000                0.1625          0.2364

   fractal_dimension_worst  Unnamed: 32
0                  0.11890          NaN
1                  0.08902          NaN
2                  0.08758          NaN
3                  0.17300          NaN
4                  0.07678          NaN
```

[94]:
```
# STEP 4: BASIC DATA CHECK

print("SHAPE:\n", df.shape)
print("\nINFO:\n")
print(df.info())
print("\nDESCRIBE:\n")
print(df.describe(include='all'))
print("\nCOLUMNS:\n")
print(df.columns)
```

```
SHAPE:
 (569, 33)

INFO:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
```

```
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
None

DESCRIBE:
```

| | id | diagnosis | radius_mean | texture_mean | perimeter_mean | \ |
|---|---|---|---|---|---|---|
| count | 5.690000e+02 | 569 | 569.000000 | 569.000000 | 569.000000 | |
| unique | NaN | 2 | NaN | NaN | NaN | |
| top | NaN | B | NaN | NaN | NaN | |
| freq | NaN | 357 | NaN | NaN | NaN | |
| mean | 3.037183e+07 | NaN | 14.127292 | 19.289649 | 91.969033 | |
| std | 1.250206e+08 | NaN | 3.524049 | 4.301036 | 24.298981 | |
| min | 8.670000e+03 | NaN | 6.981000 | 9.710000 | 43.790000 | |
| 25% | 8.692180e+05 | NaN | 11.700000 | 16.170000 | 75.170000 | |
| 50% | 9.060240e+05 | NaN | 13.370000 | 18.840000 | 86.240000 | |
| 75% | 8.813129e+06 | NaN | 15.780000 | 21.800000 | 104.100000 | |
| max | 9.113205e+08 | NaN | 28.110000 | 39.280000 | 188.500000 | |

| | area_mean | smoothness_mean | compactness_mean | concavity_mean | \ |
|---|---|---|---|---|---|
| count | 569.000000 | 569.000000 | 569.000000 | 569.000000 | |
| unique | NaN | NaN | NaN | NaN | |
| top | NaN | NaN | NaN | NaN | |
| freq | NaN | NaN | NaN | NaN | |
| mean | 654.889104 | 0.096360 | 0.104341 | 0.088799 | |
| std | 351.914129 | 0.014064 | 0.052813 | 0.079720 | |
| min | 143.500000 | 0.052630 | 0.019380 | 0.000000 | |
| 25% | 420.300000 | 0.086370 | 0.064920 | 0.029560 | |
| 50% | 551.100000 | 0.095870 | 0.092630 | 0.061540 | |
| 75% | 782.700000 | 0.105300 | 0.130400 | 0.130700 | |
| max | 2501.000000 | 0.163400 | 0.345400 | 0.426800 | |

| | concave points_mean | symmetry_mean | fractal_dimension_mean | \ |
|---|---|---|---|---|
| count | 569.000000 | 569.000000 | 569.000000 | |
| unique | NaN | NaN | NaN | |
| top | NaN | NaN | NaN | |
| freq | NaN | NaN | NaN | |
| mean | 0.048919 | 0.181162 | 0.062798 | |
| std | 0.038803 | 0.027414 | 0.007060 | |
| min | 0.000000 | 0.106000 | 0.049960 | |
| 25% | 0.020310 | 0.161900 | 0.057700 | |
| 50% | 0.033500 | 0.179200 | 0.061540 | |

| | | | | |
|---|---|---|---|---|
| 75% | 0.074000 | 0.195700 | | 0.066120 |
| max | 0.201200 | 0.304000 | | 0.097440 |

| | radius_se | texture_se | perimeter_se | area_se | smoothness_se | \ |
|---|---|---|---|---|---|---|
| count | 569.000000 | 569.000000 | 569.000000 | 569.000000 | 569.000000 | |
| unique | NaN | NaN | NaN | NaN | NaN | |
| top | NaN | NaN | NaN | NaN | NaN | |
| freq | NaN | NaN | NaN | NaN | NaN | |
| mean | 0.405172 | 1.216853 | 2.866059 | 40.337079 | 0.007041 | |
| std | 0.277313 | 0.551648 | 2.021855 | 45.491006 | 0.003003 | |
| min | 0.111500 | 0.360200 | 0.757000 | 6.802000 | 0.001713 | |
| 25% | 0.232400 | 0.833900 | 1.606000 | 17.850000 | 0.005169 | |
| 50% | 0.324200 | 1.108000 | 2.287000 | 24.530000 | 0.006380 | |
| 75% | 0.478900 | 1.474000 | 3.357000 | 45.190000 | 0.008146 | |
| max | 2.873000 | 4.885000 | 21.980000 | 542.200000 | 0.031130 | |

| | compactness_se | concavity_se | concave points_se | symmetry_se | \ |
|---|---|---|---|---|---|
| count | 569.000000 | 569.000000 | 569.000000 | 569.000000 | |
| unique | NaN | NaN | NaN | NaN | |
| top | NaN | NaN | NaN | NaN | |
| freq | NaN | NaN | NaN | NaN | |
| mean | 0.025478 | 0.031894 | 0.011796 | 0.020542 | |
| std | 0.017908 | 0.030186 | 0.006170 | 0.008266 | |
| min | 0.002252 | 0.000000 | 0.000000 | 0.007882 | |
| 25% | 0.013080 | 0.015090 | 0.007638 | 0.015160 | |
| 50% | 0.020450 | 0.025890 | 0.010930 | 0.018730 | |
| 75% | 0.032450 | 0.042050 | 0.014710 | 0.023480 | |
| max | 0.135400 | 0.396000 | 0.052790 | 0.078950 | |

| | fractal_dimension_se | radius_worst | texture_worst | perimeter_worst | \ |
|---|---|---|---|---|---|
| count | 569.000000 | 569.000000 | 569.000000 | 569.000000 | |
| unique | NaN | NaN | NaN | NaN | |
| top | NaN | NaN | NaN | NaN | |
| freq | NaN | NaN | NaN | NaN | |
| mean | 0.003795 | 16.269190 | 25.677223 | 107.261213 | |
| std | 0.002646 | 4.833242 | 6.146258 | 33.602542 | |
| min | 0.000895 | 7.930000 | 12.020000 | 50.410000 | |
| 25% | 0.002248 | 13.010000 | 21.080000 | 84.110000 | |
| 50% | 0.003187 | 14.970000 | 25.410000 | 97.660000 | |
| 75% | 0.004558 | 18.790000 | 29.720000 | 125.400000 | |
| max | 0.029840 | 36.040000 | 49.540000 | 251.200000 | |

| | area_worst | smoothness_worst | compactness_worst | concavity_worst | \ |
|---|---|---|---|---|---|
| count | 569.000000 | 569.000000 | 569.000000 | 569.000000 | |
| unique | NaN | NaN | NaN | NaN | |
| top | NaN | NaN | NaN | NaN | |
| freq | NaN | NaN | NaN | NaN | |
| mean | 880.583128 | 0.132369 | 0.254265 | 0.272188 | |

```
std      569.356993          0.022832          0.157336         0.208624
min      185.200000          0.071170          0.027290         0.000000
25%      515.300000          0.116600          0.147200         0.114500
50%      686.500000          0.131300          0.211900         0.226700
75%     1084.000000          0.146000          0.339100         0.382900
max     4254.000000          0.222600          1.058000         1.252000

        concave points_worst  symmetry_worst  fractal_dimension_worst  \
count              569.000000      569.000000               569.000000
unique                    NaN             NaN                      NaN
top                       NaN             NaN                      NaN
freq                      NaN             NaN                      NaN
mean                 0.114606        0.290076                 0.083946
std                  0.065732        0.061867                 0.018061
min                  0.000000        0.156500                 0.055040
25%                  0.064930        0.250400                 0.071460
50%                  0.099930        0.282200                 0.080040
75%                  0.161400        0.317900                 0.092080
max                  0.291000        0.663800                 0.207500

        Unnamed: 32
count           0.0
unique          NaN
top             NaN
freq            NaN
mean            NaN
std             NaN
min             NaN
25%             NaN
50%             NaN
75%             NaN
max             NaN

COLUMNS:

Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst', 'Unnamed: 32'],
      dtype='object')
```

[95]:
```
# STEP 5: CLEAN COLUMN NAMES
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
df.columns
```

[95]:
```
Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave_points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave_points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave_points_worst',
       'symmetry_worst', 'fractal_dimension_worst', 'unnamed:_32'],
      dtype='object')
```

[96]:
```
# STEP 6: CHECK MISSING VALUES
df.isnull().sum()     # null values
```

[96]:
```
id                         0
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave_points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
compactness_se             0
concavity_se               0
concave_points_se          0
symmetry_se                0
fractal_dimension_se       0
radius_worst               0
texture_worst              0
perimeter_worst            0
area_worst                 0
smoothness_worst           0
compactness_worst          0
concavity_worst            0
```

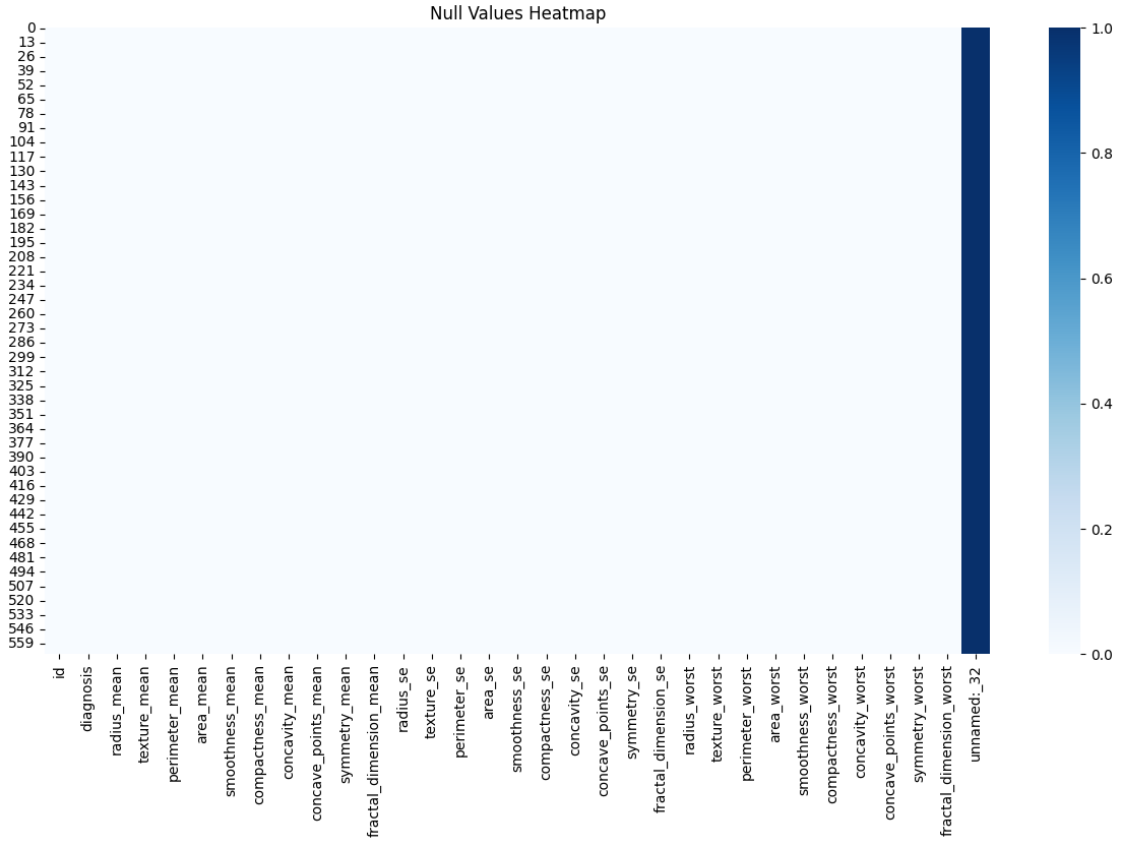
```
concave_points_worst         0
symmetry_worst               0
fractal_dimension_worst      0
unnamed:_32                569
dtype: int64
```

[97]:
```
df.notnull().sum()     # not-null values
```

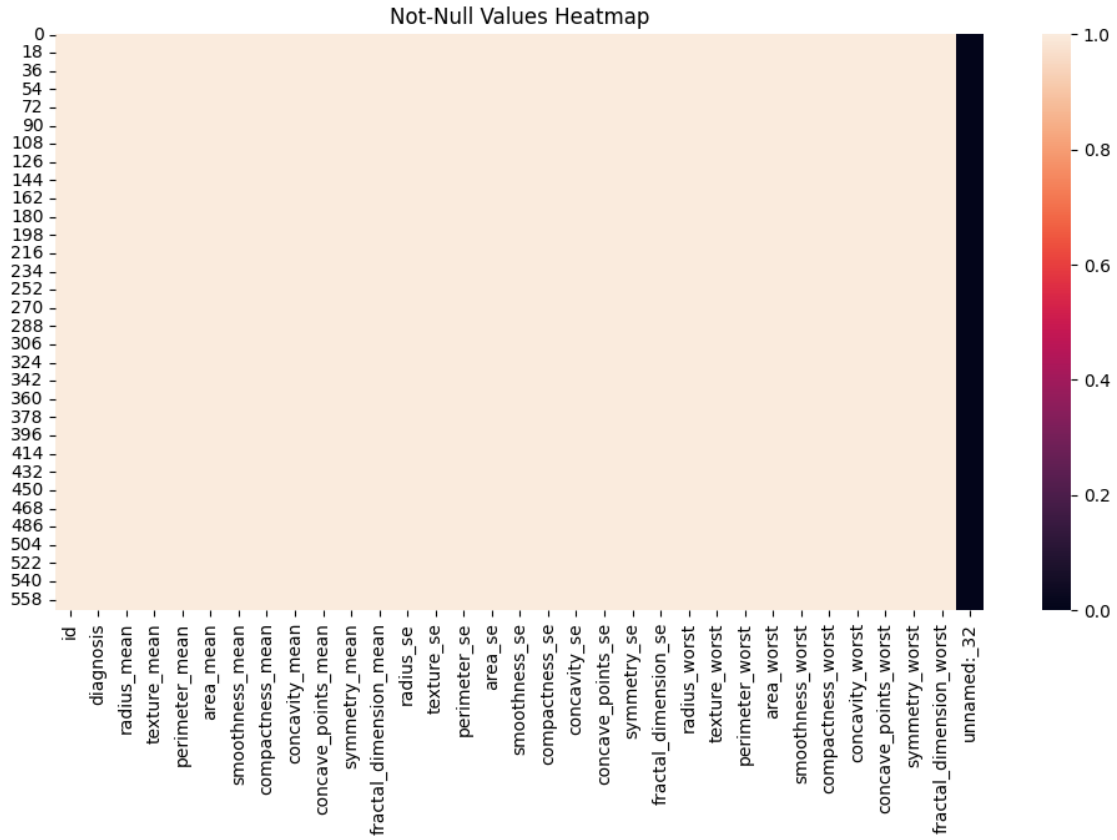
[97]:
```
id                         569
diagnosis                  569
radius_mean                569
texture_mean               569
perimeter_mean             569
area_mean                  569
smoothness_mean            569
compactness_mean           569
concavity_mean             569
concave_points_mean        569
symmetry_mean              569
fractal_dimension_mean     569
radius_se                  569
texture_se                 569
perimeter_se               569
area_se                    569
smoothness_se              569
compactness_se             569
concavity_se               569
concave_points_se          569
symmetry_se                569
fractal_dimension_se       569
radius_worst               569
texture_worst              569
perimeter_worst            569
area_worst                 569
smoothness_worst           569
compactness_worst          569
concavity_worst            569
concave_points_worst       569
symmetry_worst             569
fractal_dimension_worst    569
unnamed:_32                  0
dtype: int64
```

[98]:
```
# STEP 7: HEATMAP OF NULL VALUES
plt.figure(figsize=(15, 8))
sns.heatmap(df.isnull(), cmap="Blues", cbar=True)
plt.title("Null Values Heatmap")
```

```
plt.show()
```




[99]:
```
plt.figure(figsize=(12, 6))
sns.heatmap(df.notnull(), cbar=True)
plt.title("Not-Null Values Heatmap")
plt.show()
```




[100]:
```
# STEP 8: CHECK DUPLICATES
df.duplicated().sum()
```

[100]:
```
np.int64(0)
```

[101]:
```
# Drop duplicates
df = df.drop_duplicates()
print("Shape after removing duplicates:", df.shape)
```

```
Shape after removing duplicates: (569, 33)
```

[102]:
```
# STEP 9: CHECK & FIX DATA TYPES
df.dtypes
```

[102]:
```
id                         int64
diagnosis                 object
radius_mean              float64
texture_mean             float64
perimeter_mean           float64
area_mean                float64
smoothness_mean          float64
```

```
compactness_mean           float64
concavity_mean             float64
concave_points_mean        float64
symmetry_mean              float64
fractal_dimension_mean     float64
radius_se                  float64
texture_se                 float64
perimeter_se               float64
area_se                    float64
smoothness_se              float64
compactness_se             float64
concavity_se               float64
concave_points_se          float64
symmetry_se                float64
fractal_dimension_se       float64
radius_worst               float64
texture_worst              float64
perimeter_worst            float64
area_worst                 float64
smoothness_worst           float64
compactness_worst          float64
concavity_worst            float64
concave_points_worst       float64
symmetry_worst             float64
fractal_dimension_worst    float64
unnamed:_32                float64
dtype: object
```

[103]:
```
# STEP 10: DELETING COLUMN WITH MAX. NULL VALUES
df = df.drop(columns=['unnamed:_32'])
```

[104]:
```
# RECHECK
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   id                      569 non-null    int64
 1   diagnosis               569 non-null    object
 2   radius_mean             569 non-null    float64
 3   texture_mean            569 non-null    float64
 4   perimeter_mean          569 non-null    float64
 5   area_mean               569 non-null    float64
 6   smoothness_mean         569 non-null    float64
 7   compactness_mean        569 non-null    float64
```

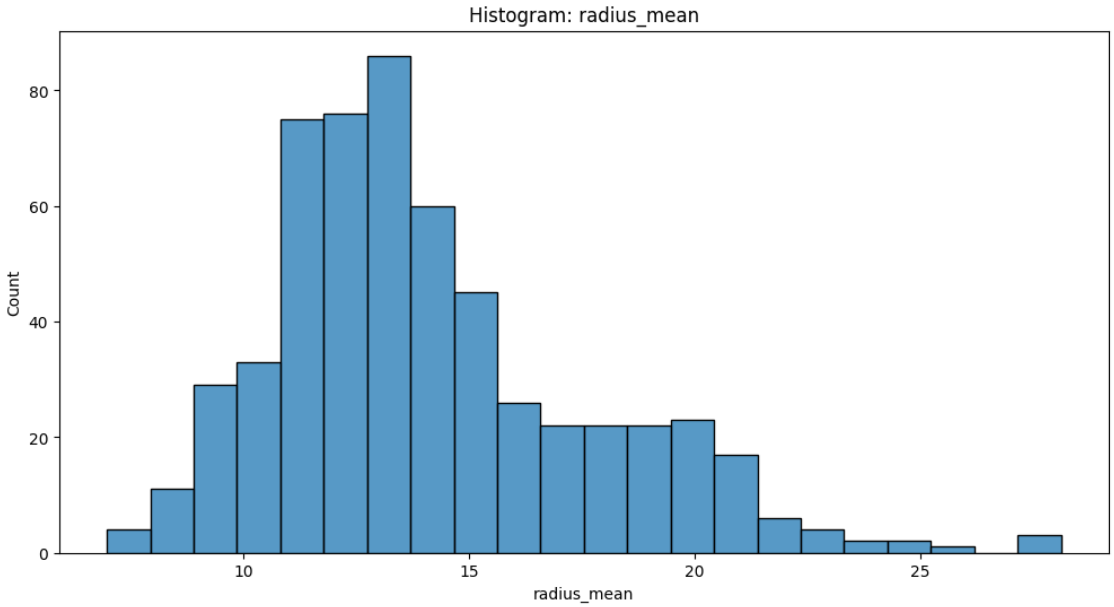
```
 8   concavity_mean           569 non-null    float64
 9   concave_points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave_points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave_points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
```

[105]:
```
# RECHECK
df.isnull().sum()      # null values
```

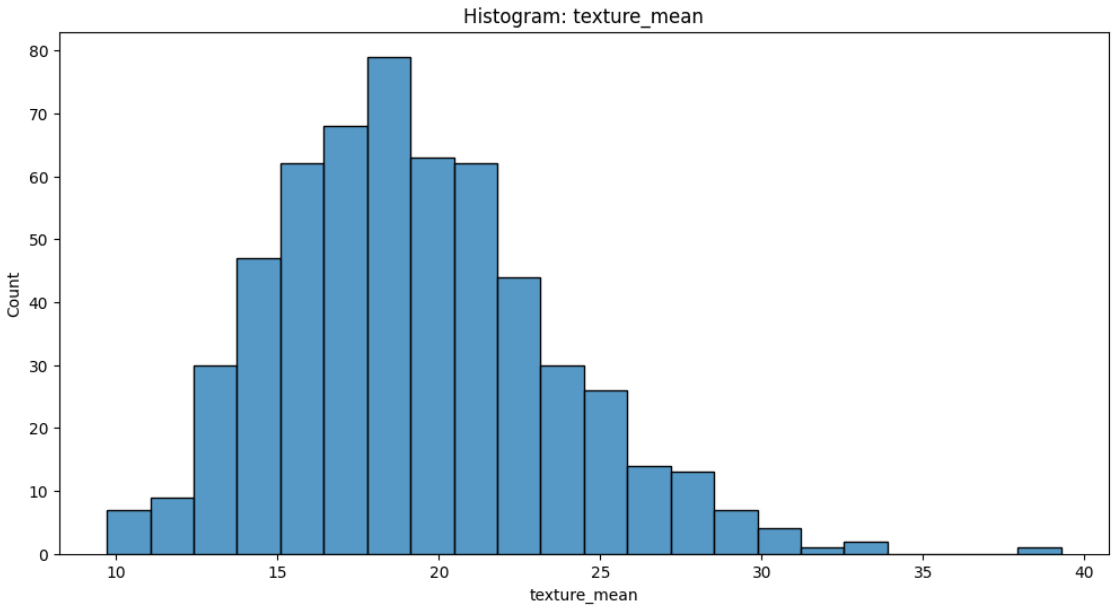
[105]:
```
id                         0
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave_points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
```

```
compactness_se             0
concavity_se               0
concave_points_se          0
symmetry_se                0
fractal_dimension_se       0
radius_worst               0
texture_worst              0
perimeter_worst            0
area_worst                 0
smoothness_worst           0
compactness_worst          0
concavity_worst            0
concave_points_worst       0
symmetry_worst             0
fractal_dimension_worst    0
dtype: int64
```

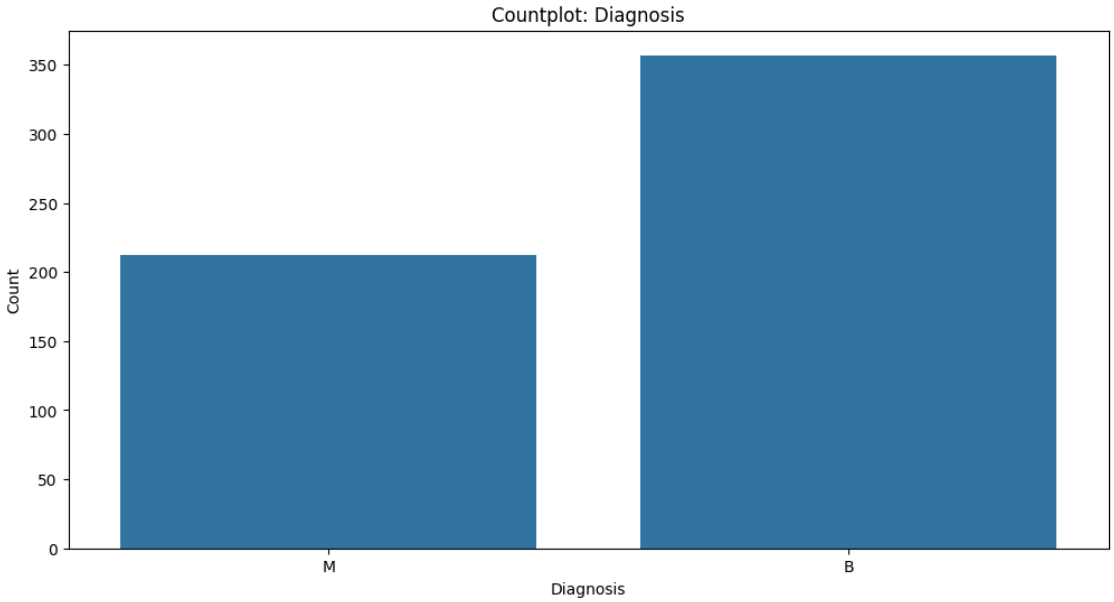
[106]:
```
# STEP 11: VISUALIZATIONS
# HISTOGRAM
plt.figure(figsize=(12,6))
sns.histplot(df['radius_mean'])
plt.xlabel("radius_mean")
plt.ylabel("Count")
plt.title("Histogram: radius_mean")
plt.show()
```



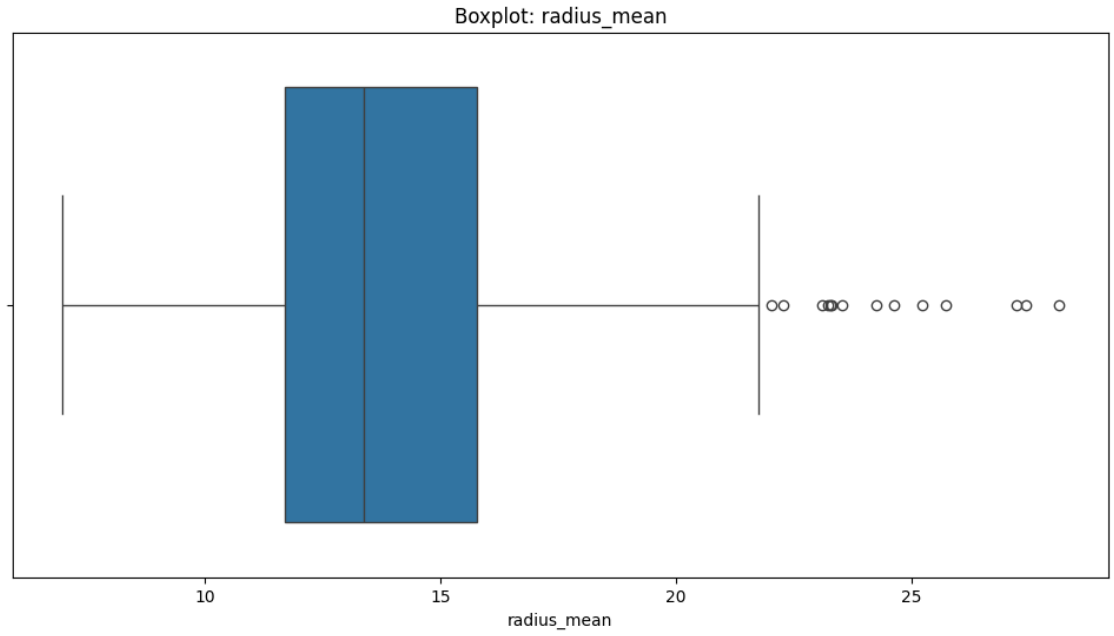
[107]:
```
# HISTOGRAM
plt.figure(figsize=(12,6))
sns.histplot(df['texture_mean'])
plt.xlabel("texture_mean")
plt.ylabel("Count")
plt.title("Histogram: texture_mean")
plt.show()
```



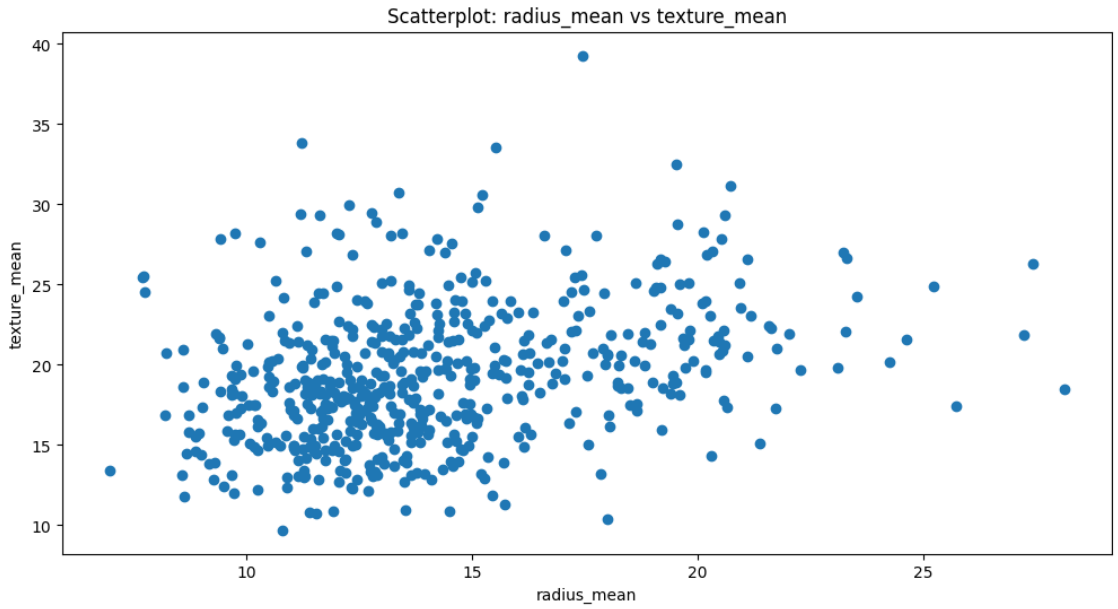

[108]:
```
# COUNTPLOT
plt.figure(figsize=(12,6))
sns.countplot(x=df['diagnosis'])
plt.xlabel("Diagnosis")
plt.ylabel("Count")
plt.title("Countplot: Diagnosis")
plt.show()
```



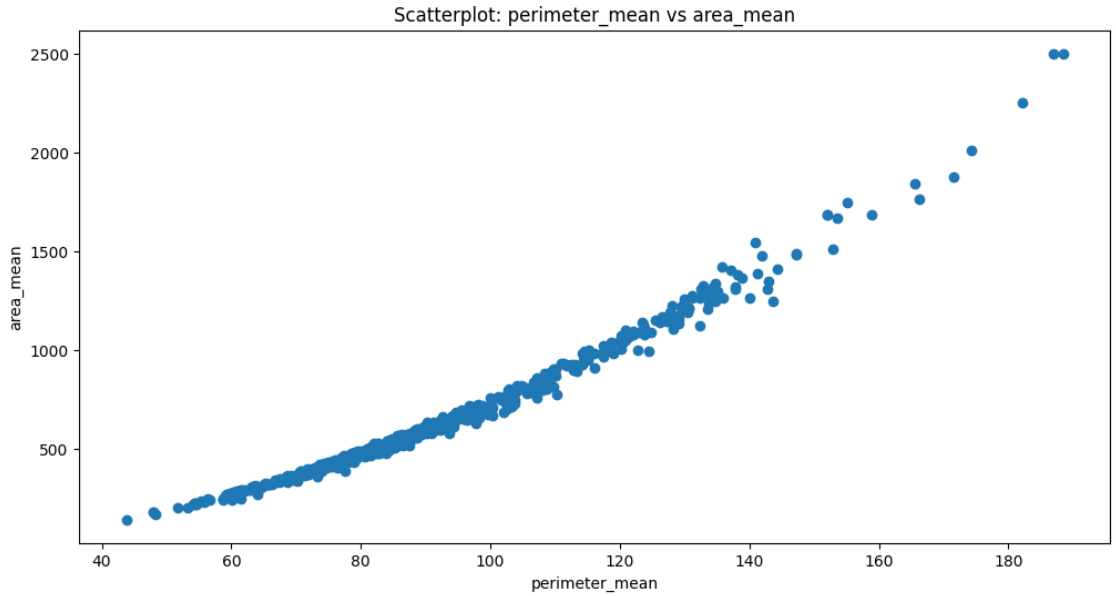

[109]:
```
# BOXPLOTS
plt.figure(figsize=(12,6))
sns.boxplot(x=df['radius_mean'])
plt.title("Boxplot: radius_mean")
plt.show()
```



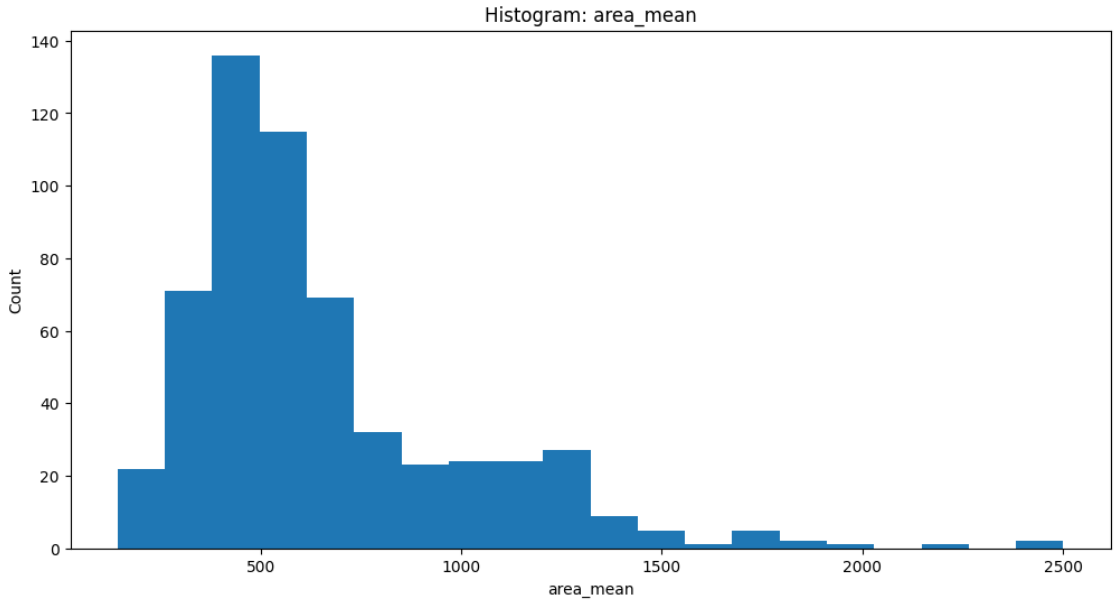
[110]:
```
# SCATTERPLOT
plt.figure(figsize=(12,6))
plt.scatter(df['radius_mean'], df['texture_mean'])
plt.xlabel("radius_mean")
plt.ylabel("texture_mean")
plt.title("Scatterplot: radius_mean vs texture_mean")
plt.show()
```



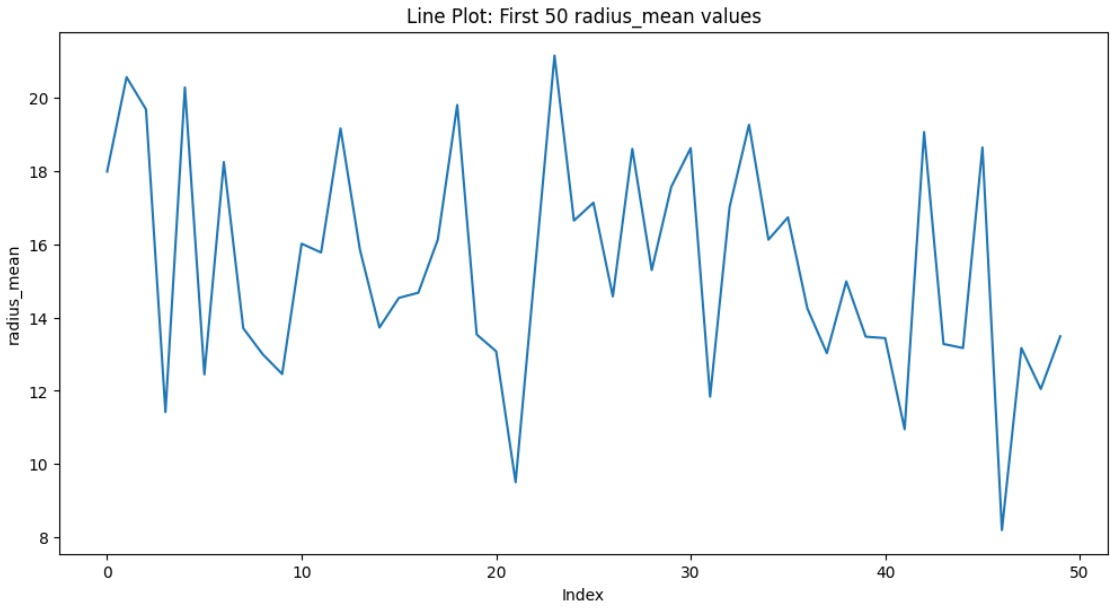

[111]:
```
# SCATTERPLOT
plt.figure(figsize=(12,6))
plt.scatter(df['perimeter_mean'], df['area_mean'])
plt.xlabel("perimeter_mean")
plt.ylabel("area_mean")
plt.title("Scatterplot: perimeter_mean vs area_mean")
plt.show()
```



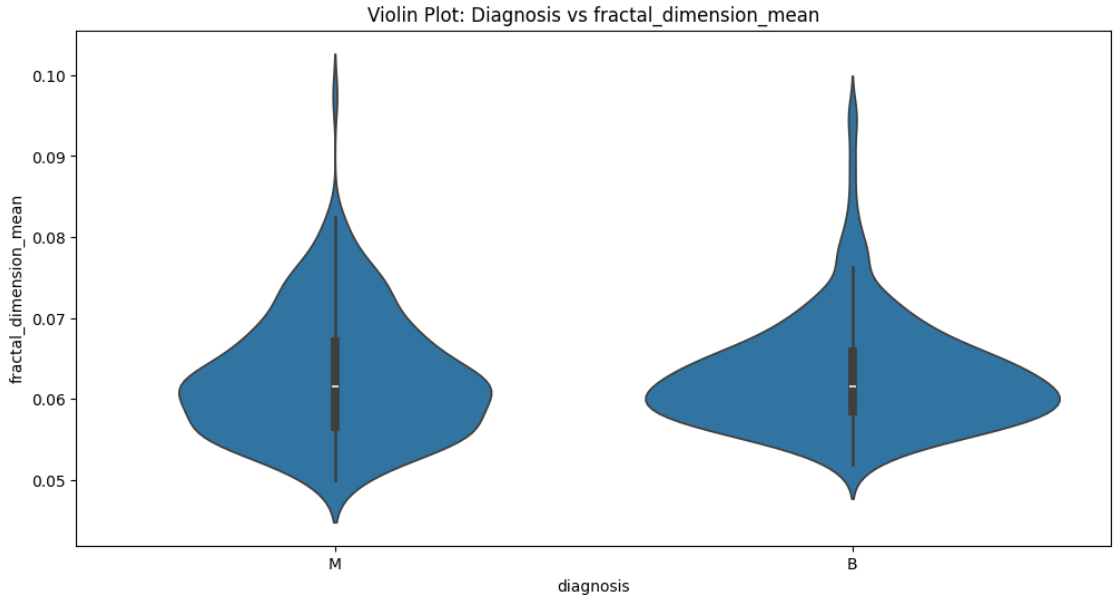

[112]:
```
# HISTOGRAM
plt.figure(figsize=(12,6))
plt.hist(df['area_mean'], bins=20)
plt.xlabel("area_mean")
plt.ylabel("Count")
plt.title("Histogram: area_mean")
plt.show()
```



[113]:
```
# LINEPLOT
plt.figure(figsize=(12,6))
plt.plot(df['radius_mean'][:50])
plt.xlabel("Index")
plt.ylabel("radius_mean")
plt.title("Line Plot: First 50 radius_mean values")
plt.show()
```



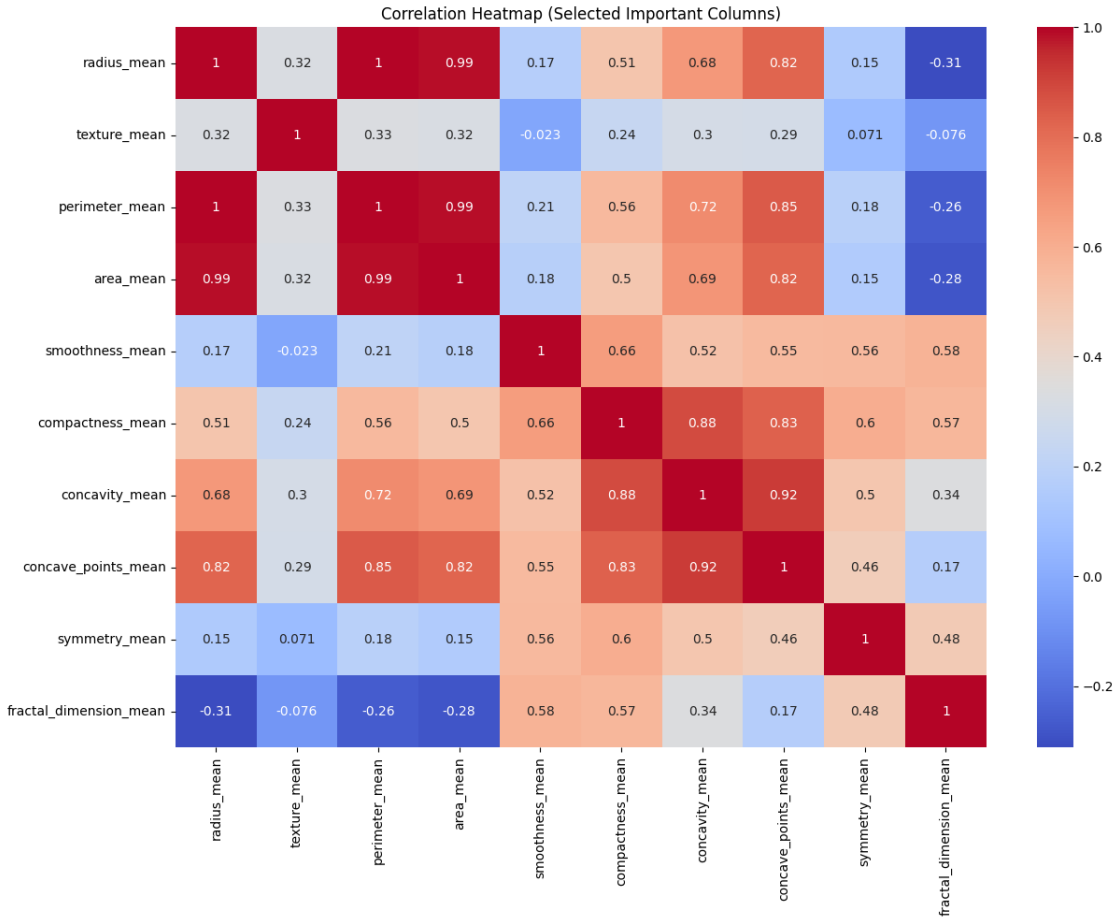

[114]:
```
#Violin Plot
plt.figure(figsize=(12,6))
sns.violinplot(x=df['diagnosis'], y=df['fractal_dimension_mean'])
plt.title("Violin Plot: Diagnosis vs fractal_dimension_mean")
plt.show()
```



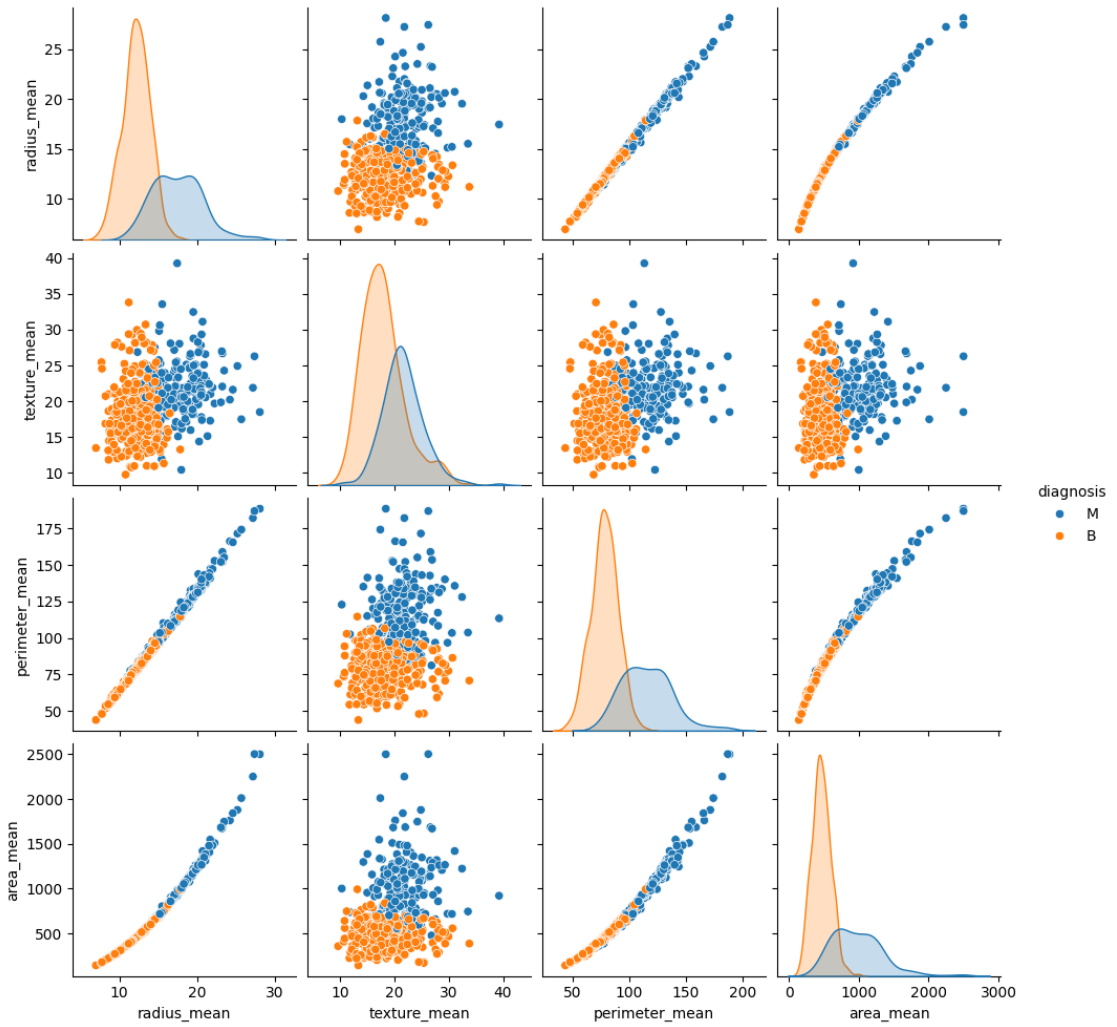

[115]:
```
# HEATMAP
plt.figure(figsize=(14,10))

corr = df[['radius_mean','texture_mean','perimeter_mean','area_mean',
           'smoothness_mean','compactness_mean','concavity_mean',
           'concave_points_mean','symmetry_mean','fractal_dimension_mean']].
 ↪corr()

sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap (Selected Important Columns)")
plt.show()
```




[116]:
```
# PAIR PLOT
sns.pairplot(df[['radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',␣
 ↪'diagnosis']], hue='diagnosis')
plt.show()
```

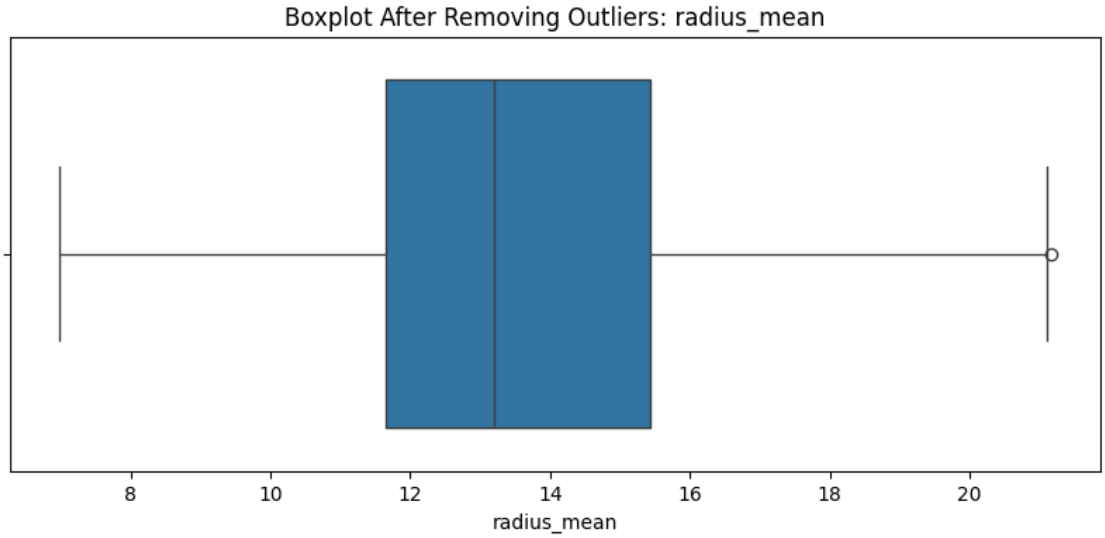



[140]:
```
# STEP: 12 HANDLING OUTLIERS OF BOX PLOT

# Function to calculate IQR and remove outliers for a given column
def cap_outliers(df_input, column_name):
  Q1 = df_input[column_name].quantile(0.25)
  Q3 = df_input[column_name].quantile(0.75)
  IQR = Q3 - Q1
  # Define Upper & Lower Limits
  lower_limit = Q1 - 1.5 * IQR
  upper_limit = Q3 + 1.5 * IQR
  # Remove Outliers
  df_output = df_input[(df_input[column_name] >= lower_limit) &␣
 ↪(df_input[column_name] <= upper_limit)]
  return df_output
```

```
# Apply the function to the 'radius_mean' column
df = cap_outliers(df, 'radius_mean')
```

[141]:
```
# Draw Boxplot Again (After Handling Outliers)
plt.figure(figsize=(10,4))
sns.boxplot(x=df['radius_mean'])
plt.title("Boxplot After Removing Outliers: radius_mean")
plt.show()
```




[121]:
```
# STEP 13:  Unique values in diagnosis
df['diagnosis'].unique()
```

[121]:
```
array(['M', 'B'], dtype=object)
```

[122]:
```
# STEP 14: # Value counts for diagnosis
df['diagnosis'].value_counts()
```

[122]:
```
diagnosis
B    357
M    198
Name: count, dtype: int64
```

[123]:
```
# STEP 15: RE CHECK
df.head()
```

[123]:

| | id | diagnosis | radius_mean | texture_mean | perimeter_mean | area_mean | \ |
|---|---|---|---|---|---|---|---|
| 0 | 842302 | M | 17.99 | 10.38 | 122.80 | 1001.0 | |
| 1 | 842517 | M | 20.57 | 17.77 | 132.90 | 1326.0 | |
| 2 | 84300903 | M | 19.69 | 21.25 | 130.00 | 1203.0 | |

```
3  84348301         M        11.42         20.38           77.58      386.1
4  84358402         M        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave_points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   symmetry_mean  fractal_dimension_mean  radius_se  texture_se  perimeter_se  \
0         0.2419                 0.07871     1.0950      0.9053         8.589
1         0.1812                 0.05667     0.5435      0.7339         3.398
2         0.2069                 0.05999     0.7456      0.7869         4.585
3         0.2597                 0.09744     0.4956      1.1560         3.445
4         0.1809                 0.05883     0.7572      0.7813         5.438

   area_se  smoothness_se  compactness_se  concavity_se  concave_points_se  \
0   153.40       0.006399         0.04904       0.05373            0.01587
1    74.08       0.005225         0.01308       0.01860            0.01340
2    94.03       0.006150         0.04006       0.03832            0.02058
3    27.23       0.009110         0.07458       0.05661            0.01867
4    94.44       0.011490         0.02461       0.05688            0.01885

   symmetry_se  fractal_dimension_se  radius_worst  texture_worst  \
0      0.03003              0.006193         25.38          17.33
1      0.01389              0.003532         24.99          23.41
2      0.02250              0.004571         23.57          25.53
3      0.05963              0.009208         14.91          26.50
4      0.01756              0.005115         22.54          16.67

   perimeter_worst  area_worst  smoothness_worst  compactness_worst  \
0           184.60      2019.0            0.1622             0.6656
1           158.80      1956.0            0.1238             0.1866
2           152.50      1709.0            0.1444             0.4245
3            98.87       567.7            0.2098             0.8663
4           152.20      1575.0            0.1374             0.2050

   concavity_worst  concave_points_worst  symmetry_worst  \
0           0.7119                0.2654          0.4601
1           0.2416                0.1860          0.2750
2           0.4504                0.2430          0.3613
3           0.6869                0.2575          0.6638
4           0.4000                0.1625          0.2364

   fractal_dimension_worst
0                  0.11890
```

```
1                0.08902
2                0.08758
3                0.17300
4                0.07678
```

## 2 STEP 2 : MODELING

[124]:
```
# STEP 1: LABEL ENCODING FOR TARGET
le = LabelEncoder()
df.loc[:, 'diagnosis'] = le.fit_transform(df['diagnosis'])
```

[125]:
```
# STEP 2: FEATURE SELECTION
X =␣
 ↪df[['radius_mean','texture_mean','perimeter_mean','area_mean','smoothness_mean',
      ␣
 ↪'compactness_mean','concavity_mean','concave_points_mean','symmetry_mean',
       'fractal_dimension_mean']]

y = df['diagnosis'].astype(int)
```

[126]:
```
# STEP 3: TRAIN-TEST SPLIT ( TRAIN= 80% AND TEST= 20% )
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)
```

[127]:
```
# STEP 4: SCALING
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

[142]:
```
# STEP 5: LOGISTIC REGRESSION
lr = LogisticRegression()
lr.fit(x_train, y_train)

y_pred_lr = lr.predict(x_test)

#Logistic Regression Training Accuracy
y_train_pred_lr = lr.predict(x_train)
train_acc_lr = accuracy_score(y_train, y_train_pred_lr)
print("Training Accuracy (LR):", train_acc_lr)


print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))
```

```
Training Accuracy (LR): 0.9459459459459459
Accuracy: 0.9009009009009009
```

```
              precision    recall  f1-score   support

           0       0.92      0.91      0.91        65
           1       0.87      0.89      0.88        46

    accuracy                           0.90       111
   macro avg       0.90      0.90      0.90       111
weighted avg       0.90      0.90      0.90       111
```

[144]:
```
# STEP 6: DECISION TREE + HYPERPARAMETER TUNING

dt = DecisionTreeClassifier()
dt.fit(x_train, y_train)
y_pred_dt = dt.predict(x_test)

#Decision Tree Training Accuracy
y_train_pred_dt = dt.predict(x_train)
train_acc_dt = accuracy_score(y_train, y_train_pred_dt)
print("Training Accuracy (DT):", train_acc_dt)

print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))

param_dt = {
    'max_depth':[3,5,7],
    'min_samples_split':[2,5,10]
}
grid_dt = GridSearchCV(DecisionTreeClassifier(), param_dt, cv=5)
grid_dt.fit(x_train, y_train)
print(grid_dt.best_params_)
print(grid_dt.best_score_)
```

```
Training Accuracy (DT): 1.0
Accuracy: 0.8828828828828829
              precision    recall  f1-score   support

           0       0.91      0.89      0.90        65
           1       0.85      0.87      0.86        46

    accuracy                           0.88       111
   macro avg       0.88      0.88      0.88       111
weighted avg       0.88      0.88      0.88       111

{'max_depth': 3, 'min_samples_split': 10}
0.9235188968335036
```

[145]:
```
# STEP 7: RANDOM FOREST + HYPERPARAMETER TUNING

rf = RandomForestClassifier()
rf.fit(x_train, y_train)
y_pred_rf = rf.predict(x_test)

#Random Forest Training Accuracy
y_train_pred_rf = rf.predict(x_train)
train_acc_rf = accuracy_score(y_train, y_train_pred_rf)
print("Training Accuracy (RF):", train_acc_rf)

print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))

param_rf = {
    'n_estimators':[50,100],
    'max_depth':[5,10,None]
}
grid_rf = GridSearchCV(RandomForestClassifier(), param_rf, cv=5)
grid_rf.fit(x_train, y_train)
print(grid_rf.best_params_)
print(grid_rf.best_score_)
```

```
Training Accuracy (RF): 1.0
Accuracy: 0.9099099099099099
              precision    recall  f1-score   support

           0       0.95      0.89      0.92        65
           1       0.86      0.93      0.90        46

    accuracy                           0.91       111
   macro avg       0.91      0.91      0.91       111
weighted avg       0.91      0.91      0.91       111

{'max_depth': 10, 'n_estimators': 50}
0.9460418794688457
```

[146]:
```
# STEP 8: KNN + HYPERPARAMETER TUNING

knn = KNeighborsClassifier()
knn.fit(x_train, y_train)
y_pred_knn = knn.predict(x_test)

#KNN Training Accuracy
y_train_pred_knn = knn.predict(x_train)
train_acc_knn = accuracy_score(y_train, y_train_pred_knn)
print("Training Accuracy (KNN):", train_acc_knn)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_knn))

param_knn = {
    'n_neighbors':[3,5,7],
    'metric':['euclidean','manhattan']
}
grid_knn = GridSearchCV(KNeighborsClassifier(), param_knn, cv=5)
grid_knn.fit(x_train, y_train)
print(grid_knn.best_params_)
print(grid_knn.best_score_)
```

```
Training Accuracy (KNN): 0.9504504504504504
Accuracy: 0.8738738738738738
              precision    recall  f1-score   support

           0       0.92      0.86      0.89        65
           1       0.82      0.89      0.85        46

    accuracy                           0.87       111
   macro avg       0.87      0.88      0.87       111
weighted avg       0.88      0.87      0.87       111

{'metric': 'euclidean', 'n_neighbors': 5}
0.9370275791624106
```

[132]:
```
# STEP 9: FINAL ACCURACY SUMMARY
print("\n FINAL MODEL ACCURACY SUMMARY")
print("Logistic Regression:", accuracy_score(y_test, y_pred_lr))
print("Decision Tree:", accuracy_score(y_test, y_pred_dt))
print("Random Forest:", accuracy_score(y_test, y_pred_rf))
print("KNN:", accuracy_score(y_test, y_pred_knn))
```

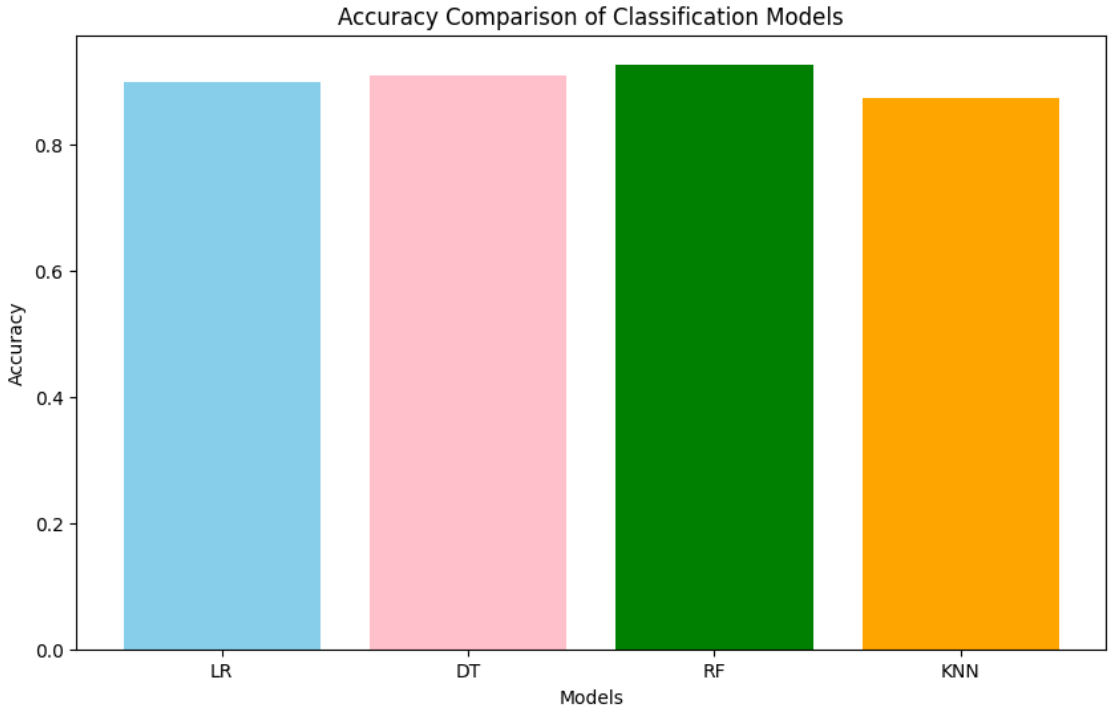
```
 FINAL MODEL ACCURACY SUMMARY
Logistic Regression: 0.9009009009009009
Decision Tree: 0.9099099099099099
Random Forest: 0.9279279279279279
KNN: 0.8738738738738738
```

[133]:
```
# STEP 10: STORE MODEL ACCURACIES

acc_lr = accuracy_score(y_test, y_pred_lr)
acc_dt = accuracy_score(y_test, y_pred_dt)
acc_rf = accuracy_score(y_test, y_pred_rf)
acc_knn = accuracy_score(y_test, y_pred_knn)
```

[134]:
```
# STEP 11: BAR GRAPH FOR MODEL ACCURACY COMPARISON
models = ['LR', 'DT', 'RF', 'KNN']
accuracies = [acc_lr, acc_dt, acc_rf, acc_knn]

plt.figure(figsize=(10,6))
plt.bar(models, accuracies, color=['skyblue','pink','green','orange'])
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.title("Accuracy Comparison of Classification Models")
plt.show()
```



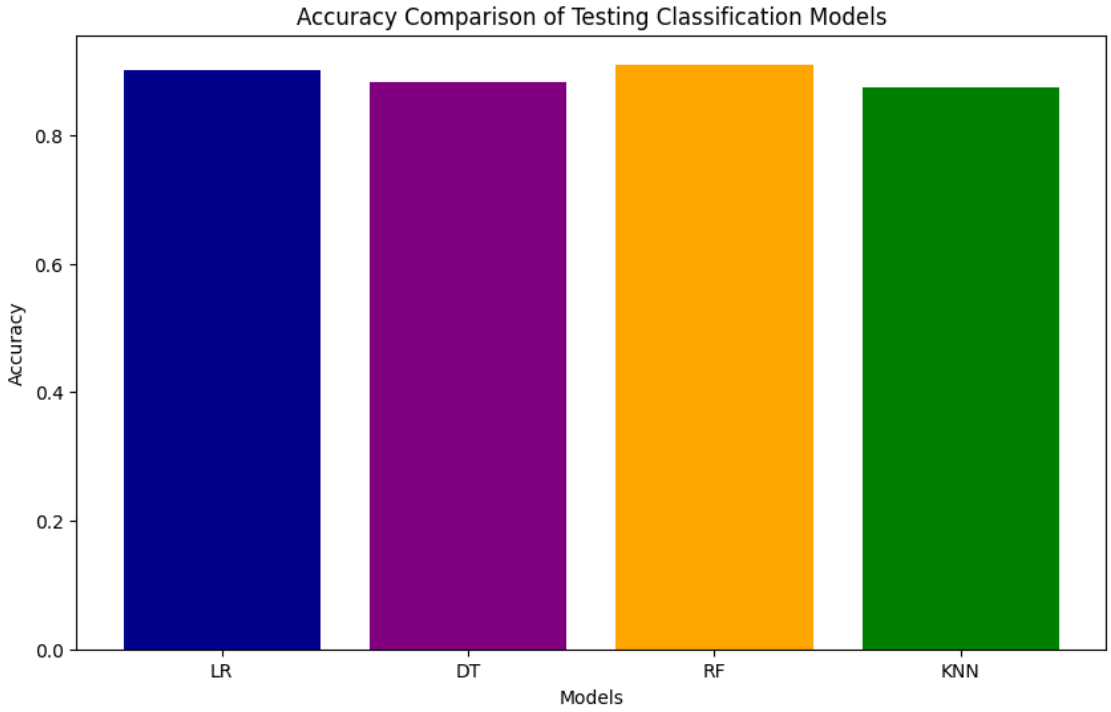

[147]:
```
# STEP 12: STORE TEST ACCURACIES
test_acc_lr = accuracy_score(y_test, y_pred_lr)
test_acc_dt = accuracy_score(y_test, y_pred_dt)
test_acc_rf = accuracy_score(y_test, y_pred_rf)
test_acc_knn = accuracy_score(y_test, y_pred_knn)

print("\nTesting Accuracies:")
print("LR:", test_acc_lr)
print("DT:", test_acc_dt)
print("RF:", test_acc_rf)
print("KNN:", test_acc_knn)
```

```
Testing Accuracies:
LR: 0.9009009009009009
DT: 0.8828828828828829
RF: 0.9099099099099099
KNN: 0.8738738738738738
```

[150]:
```
# STEP 13: BAR GRAPH FOR TEST ACCURACY COMPARISON
models = ['LR', 'DT', 'RF', 'KNN']
accuracies = [test_acc_lr,test_acc_dt, test_acc_rf,test_acc_knn]

plt.figure(figsize=(10,6))
plt.bar(models, accuracies, color=['Darkblue','purple','orange','green'])
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.title("Accuracy Comparison of Testing Classification Models")
plt.show()
```




## 3 Insights (Based on model Accuracies)

1. Decision Tree has the highest accuracy (0.9189).

-It performed the best among all four models in this dataset.

2. Logistic Regression and Random Forest have the same accuracy (0.9009).

-Both models give strong performance and are very close to the best model.

3. KNN has the lowest accuracy (0.8739).

-KNN is more sensitive to scaling and noise, so its accuracy is a bit lower.

## 4 Insights (Based on model testing Accuracies)

1.Random Forest (RF) is the best-performing model (0.9099)

-It is an ensemble model (multiple trees → better generalization)

-It reduces overfitting compared to a single Decision Tree

2.KNN performs the lowest (0.8738)

-Very sensitive to noise

[134]: